

EECS 482

Introduction to Operating Systems

Fall 2025

Prof. Peter Chen
University of Michigan
<https://os.eecs.umich.edu>



Course staff



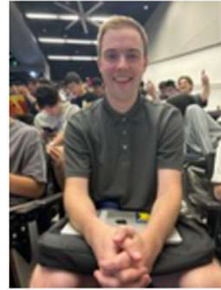
[Peter Chen](#)



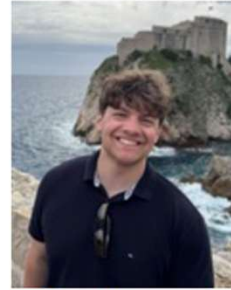
John Abraham



Hermann Bauer



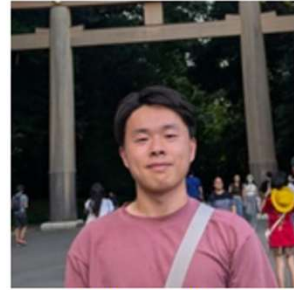
Scott Brinley



Sami Fayyad



Kelly Goss



[Wayne He](#)



Donal Higgins



[Gary Huang](#)



Sarah Hubbard



Kevin Jia



Matthew Jia



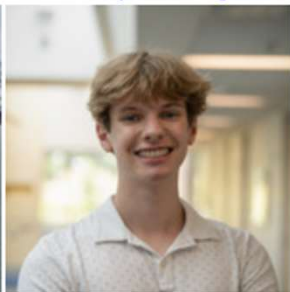
Ryan Kersten



David McDermott



Tongyuan Miao



[Mason Miller](#)



Ibrahim Musaddequr Rahman



Drishti Srivastava

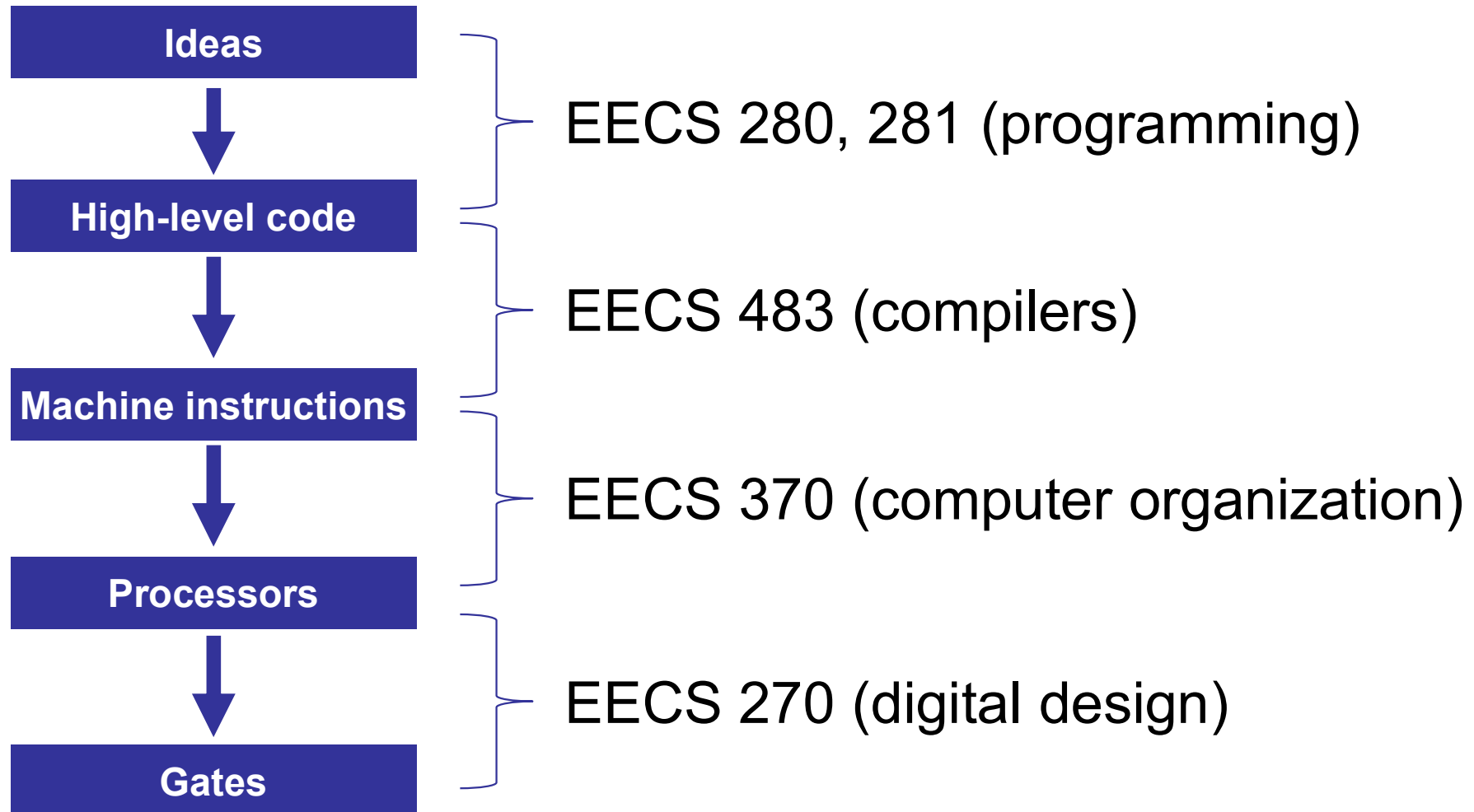


[Tiffany Zhou](#)

Agenda for today

- Why EECS 482?
 - Course syllabus and logistics
-
- Why do we need an OS and what does it do?
 - How did OSes evolve to what we have today?

Overview of CS/CE Curriculum



But some mysteries remain...

How does a computer start when you turn it on?

How to get new program into memory and start executing it?

How to ... en?

How to

How to

How to

How to

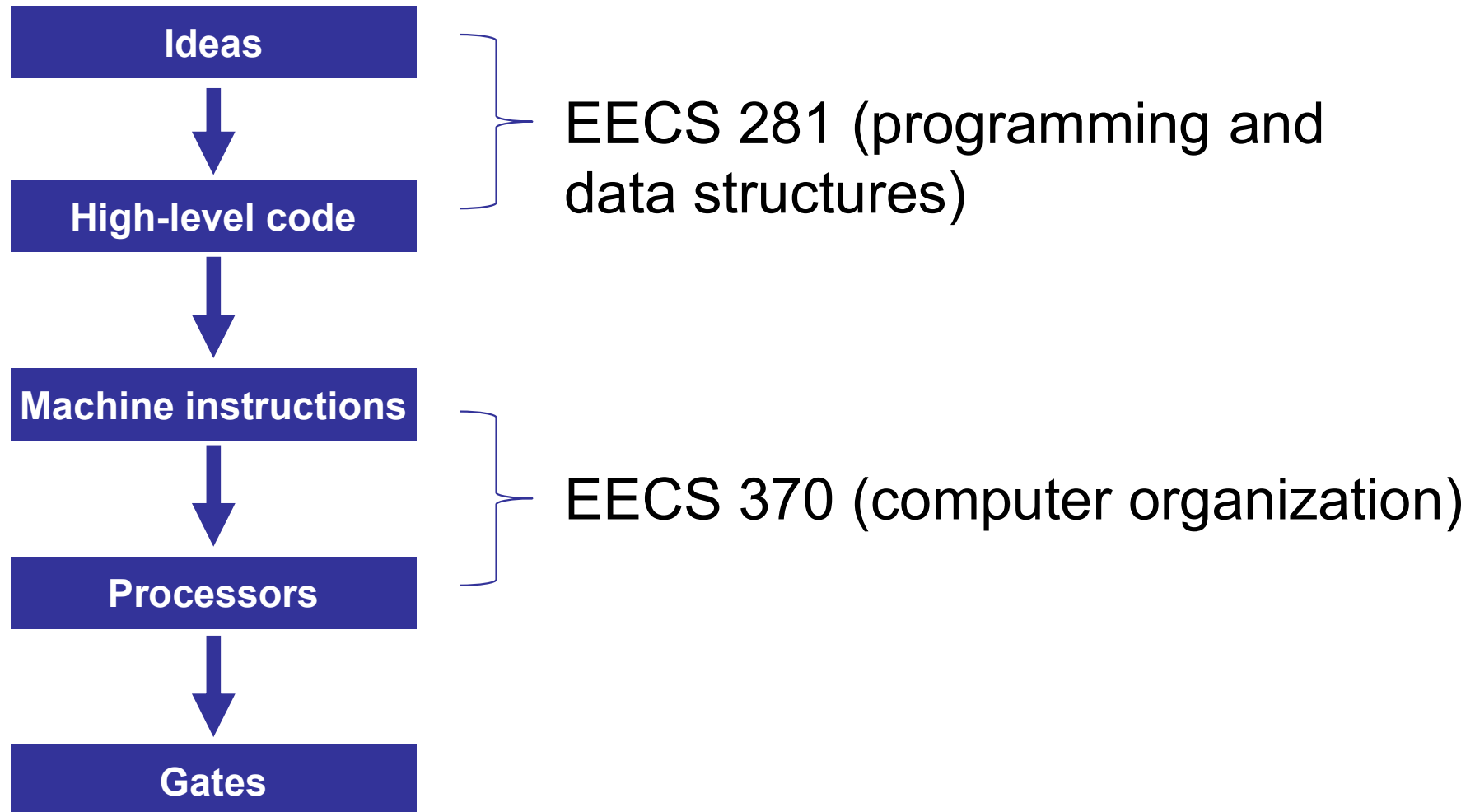
How to

How to receive e-mail when you're not on the computer?

The OS does all of this.

**After this semester, you should
be able to answer all of these
questions!**

Prerequisites



Lectures

- Lectures will be interactive
- Attend lecture and participate
 - In person
 - Via Zoom chat

Labs

Pre-lab questions will be posted on course web page

- Do them **before** going to your section

First lab section is this Friday

Course material

Course web page: <https://os.eecs.umich.edu/>

Piazza



Textbook (recommended)

- Operating Systems: Principles and Practice, 2nd edition, by Tom Anderson and Mike Dahlin

Projects

Project 0: Set up development environment

Project 1: Concurrent program

Project 2: Thread manager

Project 3: Virtual memory manager

Project 4: Multi-threaded network file system

By the end of the semester, you'll have built the main parts of an operating system

Projects

Challenging

Little code

- My total code = 946 lines

Lots of thinking

- Two views: user-level software vs. hardware
- Concurrent threads running on multiple processors
- Multiple address spaces spread across memory and disk
- Persistent data structures
- Distributed computing

Why do we assign projects in CS?

4-credits vs. 6-credits

What's the same?

- Lectures
- Lab sections
- Project 1
- Exams (probably)

What's different?

- Class time for EECS 408 structured as extra office hours
- 6-credit students implement advanced features in Projects 2-4

4-credits vs. 6-credits

	Core project	Advanced features
Project 1		
Project 2	1 CPU	> 1 CPU
Project 3	Process starts with empty arena	Process starts with copy of parent's arena
Project 4	Reader/writer locks Statically allocated locks	Upgradable reader/writer locks Dynamically allocated locks

4-credits vs. 6-credits

	Core project	Advanced features
Project 1	114 LOC	
Project 2	1 CPU 123 LOC	> 1 CPU
Project 3	Process starts with empty arena 234 LOC	Process starts with copy of parent's arena
Project 4	Reader/writer locks Statically allocated locks 402 LOC	Upgradable reader/writer locks Dynamically allocated locks
Total	873 LOC	

4-credits vs. 6-credits

	Core project	Advanced features
Project 1	114 LOC	114 (+0) LOC
Project 2	1 CPU 123 LOC	> 1 CPU 148 (+25) LOC
Project 3	Process starts with empty arena 234 LOC	Process starts with copy of parent's arena 256 (+22) LOC
Project 4	Reader/writer locks Statically allocated locks 402 LOC	Upgradable reader/writer locks Dynamically allocated locks 428 (+26) LOC
Total	873 LOC	946 (+73) LOC

Project groups

Projects 2-4 done in groups of 2 or 3

- Declare your group (by September 15)
- Post to Piazza if you don't know anyone

Choose group members carefully

- Check schedule, class goals, working style, etc.
- Mix of 4-credit and 6-credit students strongly discouraged

We'll evaluate each member's contributions

- Peer feedback
- git statistics

Project policies

Development

- Github: EECS 482 will provide **private** repositories. Commits should match contributions
- Must use course development methodology: VS Code/Neovim with version482 extension, autotag.sh, Makefile

Collaboration (with humans or AI)

- Okay to clarify project handout or discuss C++ syntax
- Not okay to discuss solutions
- Not okay to benefit from past solutions
- **Group members and course staff are your best source of help**

Soft deadlines

- Projects due by end of specified date (midnight ET)
- Initial project score
 - Autograder score at due date $[0,80]$ * hand-grading multiplier $[1,1.25]$
- Earn additional autograder points after due date
 - Extra points will **not** be multiplied by hand-grading score
 - Last day to submit: December 12
- Example
 - Autograder score at due date = 60
 - Hand-grading multiplier: 1.20
 - Autograder score at end of semester = 80
 - Final project score = $60 * 1.20 + (80-60) = 92$

Project tips

Start early!

Develop project incrementally

- Build a little functionality, then test, test, test, ...

Expect to spend most of your time debugging

Keep up with lecture and lab

Project tips

Make good use of staff – we are here to help

- Visit office hours
- Participate in discussion on Piazza
- Look for help in lecture, lab, and textbook

Our goal is to help **you** learn to solve your own problems (not to solve them for you)

Exams (must be taken in person)

- Midterm: October 20, 5:00-6:50 pm
- Final: December 17, 8:00-9:50 am

Mode of instruction

	In-person	Livestreamed	Recorded
Lectures	✓	✓	✓
Labs	✓	✓	✓
Office hours	✓	✓	
Exams	✓		

Administrative break

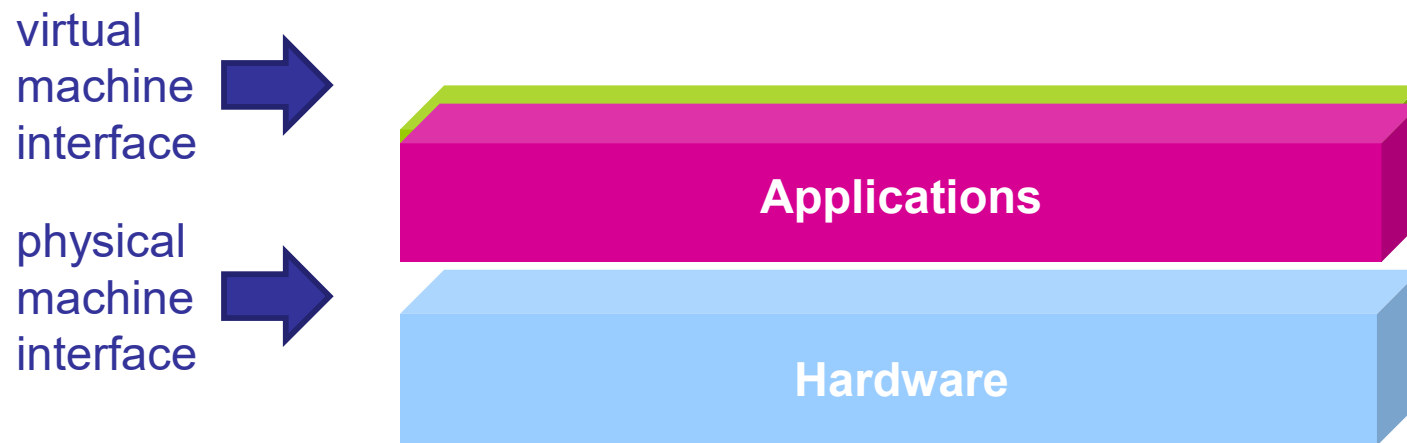
- Look for a project group
- Attend lab this Friday (Project 0 walkthrough)
- Submit your picture and (U-M linked) github username on course web page

<https://os.eecs.umich.edu>



What is an OS?

- The operating system is the software layer between application programs and the hardware



Roles of the OS: abstractions

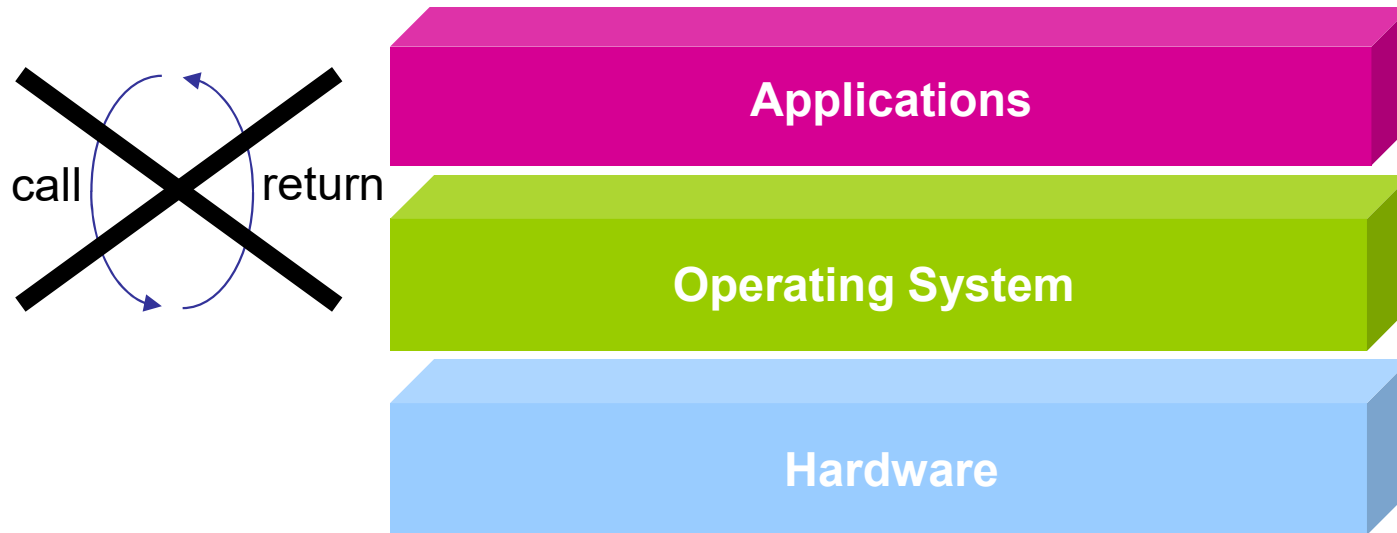
- Create **abstractions** to ease use of hardware
 - CPU → threads
 - Memory → address space
 - Disk → file system
 - Network card → sockets
- What don't you need to worry about when you program?
- For any area of OS, ask
 - What interface does hardware present to software?
 - What interface does OS present to applications?

Roles of the OS: resource management

- Manage **shared** hardware resources
 - Processor
 - Memory
 - Disk
 - Network
- Goals
 - Safety
 - Efficiency
 - Fairness

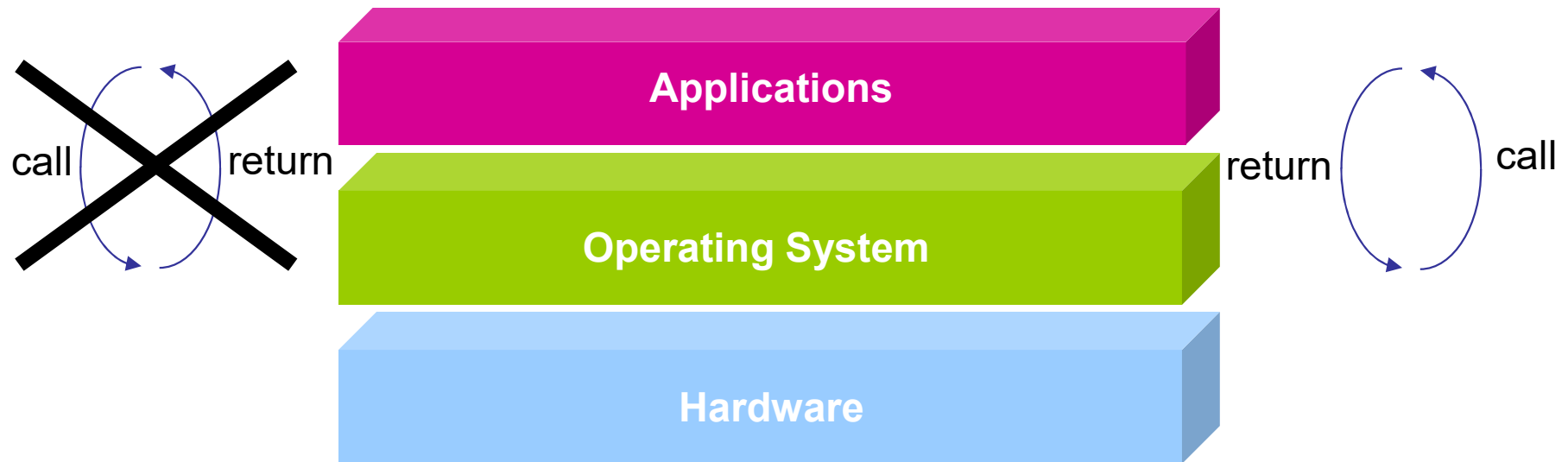
OS and apps: perspective 1

- **Application** is main program
 - Gets services by calling kernel (OS)
 - Example: print output to the screen
- This seems right, but there are some problems



OS and apps: perspective 2

- **OS** is main program
 - Calls applications as subroutines
 - Application runs until it returns control back to OS
- Who is calling whom?



Why learn about operating systems?

- You may write one someday
- Understanding what you use
 - Understanding the OS helps you write better apps
 - Functionality, performance tuning, simplicity, etc.
- Universal abstractions, techniques, optimizations
 - Concurrency, caching, indirection, naming, atomicity, protection,...
 - Examples: Cloud computing, web services, hypervisors, device drivers

History of operating systems

- Computing hardware was very expensive
- OS were very simple, then became more sophisticated to use the hardware more efficiently

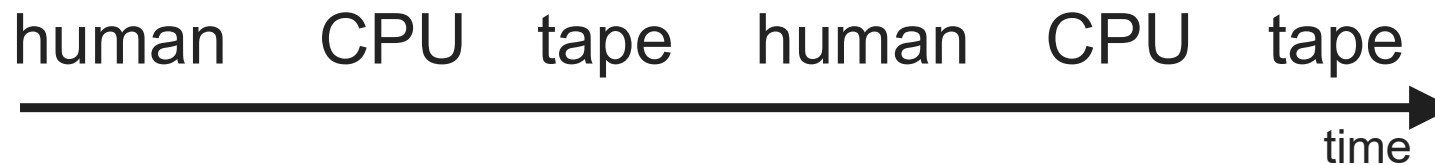


MIDAC (1953)

<https://cse.engin.umich.edu/about/history/>

Single operator at console

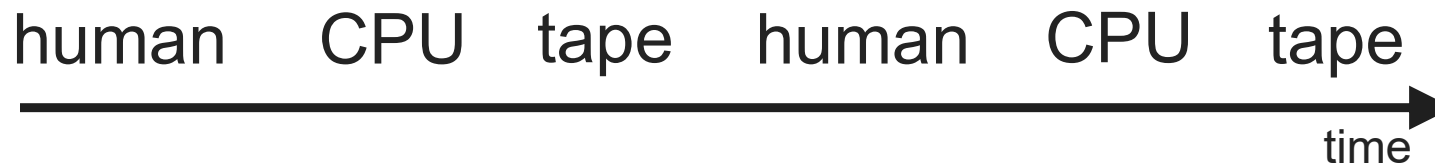
- Goal: basic functionality
- OS is library of standard services



- One thing happening at a time
 - + Interactive
 - Poor utilization of hardware
- How to improve utilization?

Batch processing

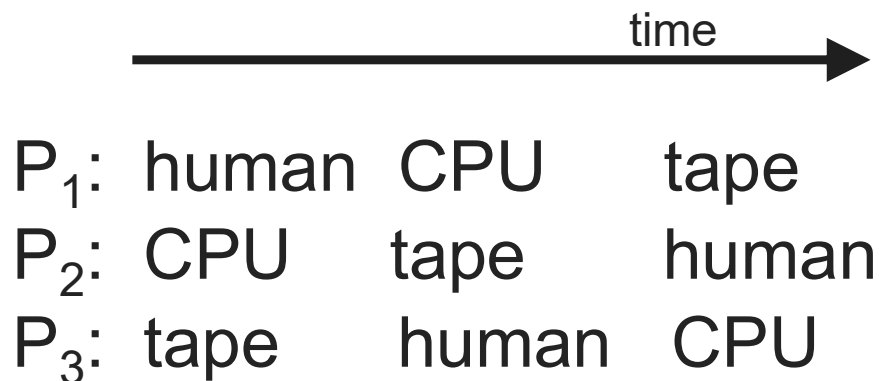
- Goal: Improve utilization by removing user interaction



- Non-interactive: submit job to **queue**, wait for completion
- OS is library of standard services + **batch monitor**
- Protection becomes an issue
 - Must ensure batch monitor is not corrupted by program
 - **Why wasn't this an issue for single operator at console?**

Time sharing

- Goals: improve utilization by overlapping CPU and I/O from different jobs; restore interactivity for humans
- Insight: switch between processes while waiting for I/O or user



- OS becomes more complex
 - Multiple jobs running concurrently
 - Multiple I/Os can take place simultaneously
 - Must protect jobs from each other

History of operating systems

- OS started out very simple, then became complex to use the (expensive!) hardware efficiently
- But hardware is no longer expensive! Do we still need these OS advances?
 - With cheap processors, do we need to run concurrent jobs?
 - With single-user systems, do we still need protection?
- Trends in operating systems for personal devices

Operating systems continue to evolve

- Smartphones: Android, iOS, ...
- Servers: virtual machines and hypervisors
- Cloud: Amazon EC2, Microsoft Azure, Google Cloud, ...